

System Design Write-Up — LastMile Delivery Tracker

Name: Anshuman
Registration Number: 23BCE1717
Email: atulvatsamishra@gmail.com

Submission for: [Unthinkable Solutions — Last-Mile Delivery Tracker Challenge](#)

This document covers the rate engine, zone detection, auto-assignment, and failed-delivery handling. A deeper technical reference is available in [ENGINEERING_DEEP_DIVE.md](#) in the project repository.

1. Rate Calculation Engine

The rate engine is a pure function (`calculateCharge`) that takes pickup zone, drop zone, dimensions, actual weight, order type, and payment type, and returns a full charge breakdown. Nothing about pricing is hardcoded — every number comes from the database.

Volumetric weight = $(L \times B \times H) / 5000$, the standard dimensional-weight divisor.

Billable weight = `max(actualWeight, volumetricWeight)`, rounded up to the nearest 0.5 kg.

The engine sets `isIntraZone = (pickupZoneId === dropZoneId)`, then looks up the matching `rate_cards` row filtered by `fromZoneId`, `toZoneId`, `orderType` (B2B/B2C), and `isIntraZone`, ordered by `effectiveFrom DESC` and constrained to `effectiveFrom <= today`. This returns the rate card that most recently became effective, so an admin can schedule a future rate change without touching code or breaking existing orders. A unique constraint on `(fromZoneId, toZoneId, orderType, isIntraZone, effectiveFrom)` keeps the lookup unambiguous.

`baseCharge = billableWeight × baseRatePerKg`. If `paymentType === COD`, a `codSurchargePercent` — configured per rate card, differing between B2B and B2C — applies on top. If no rate card matches, the engine throws `RATE_CARD_NOT_FOUND` naming the actual zones, rather than defaulting to zero. The breakdown is shown to the customer *before* confirmation via a public `POST /api/orders/calculate-charge` endpoint, and the same function runs again at order creation, so the quoted and charged prices can never drift apart.

2. Zone Detection Approach

Zones are geographic groupings (e.g. North, South, East, West) that admins manage independently of any address format. Each zone owns a set of `zone_areas` rows — free-text locality names or pincodes added through the dashboard. This many-areas-to-one-zone design means onboarding a new locality is a data operation, not a code change.

Detection happens via `GET /api/zones/detect?area=<string>`, a case-insensitive partial match (`ILIKE '%area%'`) against `zone_areas.area_name`. On the order form, as the customer types a pickup or drop address, the frontend calls this endpoint and shows the resolved zone inline ("Pickup Zone: South Zone ✓") before charge calculation runs, so mismatches are caught before submission. If multiple areas match, all candidates are returned with a `confidence` flag (`HIGH` for a single match, `MULTIPLE_MATCHES` otherwise), so ambiguity is surfaced rather than silently resolved wrong.

This is intentionally simple for the MVP. The natural extension is `pg_trgm` trigram similarity for typo tolerance (e.g. "Bandara" still resolving to "Bandra"), or a postal-code/geocoding API for exact resolution at scale — both are additive on top of the existing `zone_areas` table, not a redesign.

3. Auto-Assignment Logic

`assignNearestAgent(orderId)` implements zone-first matching: it queries `agent_availability` for agents whose `currentZoneId` matches the order's `pickupZoneId` and `isAvailable = true`, breaking ties FIFO by `lastUpdated` (longest-idle agent first, balancing load). If no zone match exists, it falls back to any available agent system-wide rather than leaving the order unassigned. If none exist at all, it throws `NO_AGENT_AVAILABLE` (409) so an admin can assign manually — the system never silently fails to assign.

On success, the assignment runs in a single transaction: the order's `assignedAgentId` is set, the agent's `isAvailable` flips to `false`, and an immutable tracking event is inserted recording who was assigned and why (zone match vs. fallback). When an order reaches a terminal status (`DELIVERED` or `FAILED`), the agent is automatically released back to `isAvailable = true` — without this, agents would accumulate as permanently "busy" after their first delivery.

Zone-based matching is a deliberate MVP decision, not an overlooked limitation: real GPS-based nearest-agent assignment needs live agent coordinates and a PostGIS `ST_Distance` query against them, a drop-in replacement for step one once agents report location. Zone centroids (`latitude` / `longitude` already on the `zones` table) stand in for this on the admin map view today.

4. Failed Delivery Handling

When an agent marks an order `FAILED`, the standard status-update flow runs: the transition is validated against the status machine, an immutable `order_tracking_events` row is inserted, the assigned agent is released, and the customer receives an email with a "Reschedule Delivery" button linking to `/track/:trackingNumber/reschedule`.

On that page the customer picks a new date (minimum tomorrow) and a reason. The reschedule service enforces three guards: the order must currently be `FAILED`; no existing reschedule may already target a future date for this order — idempotency, so a double-submit can't create two pending reschedules, rejected with `409 RESCHEDULE_ALREADY_PENDING`; and the new date must be at least tomorrow. If all pass, a transaction creates the `delivery_reschedules` record, resets the order to `PENDING` with the new `scheduledDeliveryDate`, and appends a tracking event. Auto-assignment then runs again outside that transaction, so a temporary lack of agents doesn't roll back the reschedule itself, and a confirmation email is sent.

The tracking table is append-only: every status change, assignment, and reschedule is a new row, never updated or deleted. This gives a tamper-evident audit trail that reconstructs an order's exact state at any point in time — needed for customer disputes and for debugging the assignment logic after the fact.